MarketLens: Q&A Export

Document questions D1 and D2 (equations and software requirements), and system questions S1 to S6. Sources: Final ISCP Manuscript (2026) and the project folder flask_webapp_dss, as read on 26 October 2026.

How to read this guide

Every number in the equation worked examples carries a tag. Before you check a number, look for its tag to see where it came from.

Tag	What it means	Example from this guide
[IN]	Input: a value the system gets from a user, a data source or the database	Competitor count from the Google Places snapshot
[K]	Constant: a fixed number set by the researchers or written in the code	30 competitors = fully crowded (manuscript p.103)
[R]	Result: calculated from the values in the same line or the line above	CD = 0.5
[EX]	Example value: picked by me to show the arithmetic. It is NOT from the real data	A barangay with 15 competitors
[M p.X]	Manuscript page X (the List of Equations page, or the section page)	[M p.103] = Eq 1 to 4 area
[C: path]	A file in the project folder where this value or logic is written (read and verified)	[C: app/ml/constants.py]
[T]	Training report: app/ml/model_store/training_report.json, the saved results of the last training run	MAE 3.371 for the market model

Honesty note. Worked examples use made-up inputs ([EX]) so the arithmetic is easy to follow. They are not forecasts for any real barangay. The only real results are the ones tagged [T] or [M]. The pinned chat "Capstone web platform development (reprompt)" could not be read from here, so nothing in this file comes from it.

D1A. Equations, explained with a worked example

There are 35 equations. The manuscript lists them on pp.102 to 116 (Eq 1 to 33) and pp.132 to 133 (Eq 34 and 35). Page numbers below follow the manuscript's *List of Equations*. The running example is a made-up barangay: 15 competitors, 60% of businesses survive, normalized population 0.5, normalized rent 0.4, urban = yes. Think of a barangay as a street of stores, and the saturation index as a crowding meter from 0 (empty) to 100 (jammed).

Group 1: Training label for the crowding score (Eq 1 to 4)

The models learn from 4,000 synthetic records because the real DTI business records were confidential (M p.103). Each record gets a crowding 'answer key' built from these four equations.

Equation 1 · Market Saturation Index training label [M p.102]

MSI_{label} = clip(100 × (0.45·CD + 0.35·DT + 0.20·SD) + ε , 0, 100)

Symbol	Meaning	Where the value comes from
MSI_{label}	The crowding answer key for one synthetic record (0 to 100)	[R] Calculated for each training record
CD, DT, SD	The three scores from Eq 2, 3 and 4	[R] Eq 2, 3, 4 below
0.45, 0.35, 0.20	Weights: competitor count matters most, then demand, then community	[K] [C: app/ml/model_store/training_report.json] msi_weights; [M p.103]
ε	Random noise so labels are not perfectly regular	[K] Normal, mean 0, standard deviation 4 [M p.103]
clip(..., 0, 100)	Keeps the answer between 0 and 100	[K] Rule in [M p.103]

Worked example (tags show where each number comes from)

- Using CD = 0.5, DT = 0.4, SD = 0.57 (from Eq 2 to 4): 100 × (0.45×0.5 + 0.35×0.4 + 0.20×0.57) = 100 × (0.225 + 0.14 + 0.114) = **47.9** before noise.
- Noise ε is random, so the training label is about 47.9 plus or minus a few points.

Result: about 47.9 (before noise)

Why it matters: The weights are the researchers' judgment, not learned from data. They define what 'crowded' means for the models.

Equation 2 · Competitor density score [M p.103]

CD = min(1 , competitor_count ÷ 30)

Symbol	Meaning	Where the value comes from
competitor_count	Number of same-industry businesses in the barangay	[IN] Google Places snapshot; synthetic value in training
30	Count at which a barangay is treated as fully crowded	[K] [M p.103]
min(1, ...)	Caps the score at 1	[K] rule

Worked example (tags show where each number comes from)

- 15 ÷ 30 = **0.5** [EX count of 15]. Half of the 'full' count.

Result: CD = 0.5

Why it matters: More competitors push the crowding score up. A barangay with 30 or more is capped at 1.

Equation 3 · Demand trend score [M p.103]

DT = 1 − historical_success_rate

Symbol	Meaning	Where the value comes from
historical_success_rate	Share of businesses in that barangay and industry that are still operating	[IN] Market data (MarketData record); [EX] 0.60
DT	Lower survival means more crowding	[R]

Worked example (tags show where each number comes from)

- 1 − 0.60 = **0.40** [EX survival rate of 60%].

Result: DT = 0.40

Why it matters: If many businesses here have closed, that is a sign the demand is weak, so saturation goes up.

Equation 4 · Socio-demographic score [M p.103]

$SD = \min(1 , 0.5 \cdot P_{norm} + 0.3 \cdot R_{norm} + 0.2 \cdot U)$

Symbol	Meaning	Where the value comes from
P_{norm}	Population density, scaled to 0 to 1	[IN] PSA 2024 barangay population; [EX] 0.5
R_{norm}	Average rent, scaled to 0 to 1	[IN] Market data; [EX] 0.4
U	1 if the barangay is urban, 0 if not	[IN] Barangay type; [EX] 1 (urban)
0.5, 0.3, 0.2	Weights for population, rent and urban status	[K] [M p.103]

Worked example (tags show where each number comes from)

- $0.5 \times 0.5 + 0.3 \times 0.4 + 0.2 \times 1 = 0.25 + 0.12 + 0.20 =$ **0.57**.

Result: SD = 0.57

Why it matters: Dense, expensive, urban areas tend to be more competitive.

Group 2: The forest's estimate and its confidence (Eq 5 and 6)

Equation 5 · Random Forest prediction of the Market Saturation Index [M p.104]

$MSI = (1 \div T) \times \Sigma_{t=1..T} f_t(x)$

Symbol	Meaning	Where the value comes from
T	Number of trees in the forest	[K] 100 trees [C: app/ml/constants.py] N_ESTIMATORS = 100
$f_t(x)$	Estimate of tree t for this market	[R] Each tree's answer
x	The eight market features: competitor count, population density, foot traffic, rent, survival rate, business density, years in operation, industry type	[IN] See [T] feature_importances
MSI	Final average estimate (0 to 100)	[R]

Worked example (tags show where each number comes from)

- Three example trees say 40, 45 and 50. Average = (40 + 45 + 50) ÷ 3 = **45** [EX; the real system averages all 100 trees].

Result: MSI = 45 (example)

Why it matters: Averaging many trees smooths out the error of any single tree.

Equation 6 · Confidence level of a forecast [M p.104]

$Confidence = \max(40 , \min(100 , 100 - (\sigma_{trees} \div 25) \times 60))$

Symbol	Meaning	Where the value comes from
σ_{trees}	Standard deviation (spread) of the 100 tree estimates	[R] Calculated from the trees
25, 60	Scaling constants	[K] [M p.104]
40, 100	Floor and ceiling of the confidence	[K] [M p.104]

Worked example (tags show where each number comes from)

- Spread σ = 10 [EX]. 10 ÷ 25 × 60 = 24. 100 − 24 = **76%**.
- A tight spread (σ = 0) gives 100%. A very wide spread is held at a floor of 40%.

Result: Confidence = 76% (example)

Why it matters: Confidence measures agreement between trees. It is not the probability that a business will succeed.

Group 3: Level labels and clustering (Eq 7 to 9)

Equation 7 · K-Means objective function [M p.105]

$$J = \Sigma_{k=1..K} \Sigma_{x_i \in C_k} \|x_i - \mu_k\|^2$$

Symbol	Meaning	Where the value comes from
K	Number of clusters: 4 (Low, Moderate, High, Saturated)	[K] [C: app/ml/constants.py] N_CLUSTERS = 4
C_k	Records assigned to cluster k	[R] Assigned by the algorithm
μ_k	Center (centroid) of cluster k	[R] Mean of the records in the cluster
J	Total squared distance from records to their centers (the algorithm makes this smaller)	[R]

Worked example (tags show where each number comes from)

- One record 0.5 units from its center adds $0.5^2 =$ **0.25** to J [EX]. K-Means is used only during training.

Result: J is minimized during training

Why it matters: K-Means finds four natural groups of markets. The live system uses fixed cut points (Eq 9) instead, so every page uses the same labels.

Equation 8 · Euclidean distance [M p.106]

$$d(x_i, \mu_k) = \sqrt{\Sigma_{j=1..n} (x_{ij} - \mu_{kj})^2}$$

Symbol	Meaning	Where the value comes from
n	Number of measures: 2 (competitor density score and normalized business density)	[K] [M p.106]
x_{ij}, μ_{kj}	Record and centroid values for measure j	[IN] / [R]

Worked example (tags show where each number comes from)

- Record (0.5, 0.4) to center (0.2, 0.8): $\sqrt{(0.5 - 0.2)^2 + (0.4 - 0.8)^2} = \sqrt{0.09 + 0.16} = \sqrt{0.25} =$ **0.5** [EX].

Result: distance = 0.5

Why it matters: This is straight-line distance, the same as Pythagoras' theorem.

Equation 9 · Saturation level classification [M p.106]

Level = Low if MSI ≤ 25 · Moderate if 25 < MSI ≤ 50 · High if 50 < MSI ≤ 75 · Saturated if MSI > 75

Symbol	Meaning	Where the value comes from
MSI	Predicted Market Saturation Index (0 to 100)	[R] Eq 5 (or Eq 10 fallback, or Eq 12 adjusted)
25, 50, 75	Cut points, four equal bands	[K] [C: app/ml/constants.py] CLUSTER_THRESHOLDS = [25, 50, 75, 100]

Worked example (tags show where each number comes from)

- MSI = 47.9 (from Eq 1 example) lies in 25 to 50, so the label is **Moderate**.
- MSI = 80 would be **Saturated**.

Result: Moderate (for MSI 47.9)

Why it matters: This turns a number into a word a non-technical owner can act on.

Group 4: Fallback and sub-category adjustment (Eq 10 to 12)

Equation 10 · Formula-based fallback for the saturation index [M p.107]

$$MSI = clip(100 \times [w_1 \cdot min(1, cc \div 30) + w_2 \cdot (1 - hsr) + w_3 \cdot min(1, bd \div 10)] , 0, 100)$$

Symbol	Meaning	Where the value comes from
cc	Competitor count	[IN] Google Places snapshot; [EX] 15
hsr	Historical success rate	[IN] Market data; [EX] 0.60
bd	Business density of the barangay	[IN] LGU data, or a placeholder if no upload exists; [EX] 5
w_1, w_2, w_3	Weights 0.45, 0.35, 0.20; editable by the Admin	[K] System Settings; [C: app/ml/model_store/training_report.json]; [M p.107]
30, 10	Saturation points for competitors and business density	[K] [M p.107]

Worked example (tags show where each number comes from)

- Competitor part: 0.45 × min(1, 15÷30) = 0.45 × 0.5 = 0.225
- Demand part: 0.35 × (1 − 0.60) = 0.35 × 0.4 = 0.14
- Community part: 0.20 × min(1, 5÷10) = 0.20 × 0.5 = 0.10
- Total: 100 × (0.225 + 0.14 + 0.10) = **46.5** [EX].

Result: MSI = 46.5 (example)

Why it matters: The system always gives an answer. If the trained model cannot load, this formula is used and reported at a fixed 50% confidence [M p.107].

Equation 11 · Sub-category competition ratio [M p.108]

$$r = clamp(D_{measured} \div (cc \times s) , 0.25, 2.5)$$

Symbol	Meaning	Where the value comes from
$D_{measured}$	Direct competitors of the sub-category (for example bakeries) found in the barangay	[IN] Google Places count; [EX] 6
cc	Competitor count for the whole industry	[IN] [EX] 15
s	Expected share of the sub-category within the industry	[K] Share table, see [C: app/ml/subcategories.py]; [EX] 0.2
clamp(…, 0.25, 2.5)	Stops one unusual count from distorting the result	[K] [M p.108]

Worked example (tags show where each number comes from)

- Expected bakeries = cc × s = 15 × 0.2 = 3 [R].
- r = 6 ÷ 3 = **2.0** [EX]. Within the 0.25 to 2.5 limits.

Result: r = 2.0 (example)

Why it matters: A plan's niche can be more or less crowded than its whole industry. This ratio captures that difference.

Equation 12 · Adjusted competitor count [M p.108]

$$cc_{adj} = round(cc \times r)$$

Symbol	Meaning	Where the value comes from
cc_{adj}	Adjusted competitor count used for the plan	[R]
r	Ratio from Eq 11	[R]

Worked example (tags show where each number comes from)

- round(15 × 2.0) = round(30) = **30** [EX]. The adjusted count is fed back into the forest to get MSI*.

Result: cc_{adj} = 30 (example)

Why it matters: MSI is the saturation figure used for the plan from this point on.*

Group 5: Plan money (Eq 13 to 15)

Equation 13 · Residents per business [M p.109]

RPB = pop ÷ (cc + 1)

Symbol	Meaning	Where the value comes from
pop	Population of the barangay	[IN] 2024 PSA figure; [M p.44] and [C: app/ml/seed_data.py] (get_real_population); [EX] 10,000
cc	Competitor count	[IN] [EX] 15
+1	Stops a division by zero when there are no competitors	[K]

Worked example (tags show where each number comes from)

- 10,000 ÷ (15 + 1) = 10,000 ÷ 16 = **625** [EX].

Result: RPB = 625 residents per business (example)

Why it matters: Measures how many potential customers each business shares.

Equation 14 · Monthly fixed cost [M p.109]

MFC = max(1,000 , rent + emp × wage × 26)

Symbol	Meaning	Where the value comes from
rent	Monthly rent of the location	[IN] Market data; [EX] ₱8,000
emp	Number of employees	[IN] Plan form; [EX] 2
wage	Daily minimum wage	[K] ₱590 (DOLE Wage Order No. RBIII-26) [C: app/ml/model_store/training_report.json] daily_wage_php = 590; [M p.44]
26	Working days per month	[K] [C: training_report.json] operating_days_per_month = 26
1,000	Floor for the monthly cost	[K] [C: training_report.json] monthly_fixed_cost_floor_php = 1000

Worked example (tags show where each number comes from)

- Staff cost = 2 × ₱590 × 26 = ₱30,680 [R].
- Total = ₱8,000 + ₱30,680 = **₱38,680** per month [EX].

Result: MFC = ₱38,680 (example)

Why it matters: This is the monthly bill the business must cover before it earns any profit.

Equation 15 · Capital runway [M p.109]

RW = capital ÷ MFC

Symbol	Meaning	Where the value comes from
capital	Startup capital the owner enters	[IN] Plan form; [EX] ₱500,000
MFC	Monthly fixed cost from Eq 14	[R] ₱38,680

Worked example (tags show where each number comes from)

- 500,000 ÷ 38,680 = **12.9 months** [EX].

Result: RW = 12.9 months (example)

Why it matters: Answers: how many months can the capital keep the lights on with no sales?

Group 6: Ramp-up and the sales needed (Eq 16 to 20)

Equation 16 · Market viability [M p.110]

V_m = (100 − MSI*) ÷ 10

Symbol	Meaning	Where the value comes from
MSI*	Adjusted saturation index from Eq 5 with cc_{adj}	[R] Eq 5, 12
V_m	Market viability on a 0 to 10 scale	[R]

Worked example (tags show where each number comes from)

- (100 − 46) ÷ 10 = **5.4** out of 10 [EX MSI* = 46].

Result: V_m = 5.4 (example)

Why it matters: A market with MSI 30 scores 7.0 out of 10 [M §4.3.2.7].

Equation 17 · Saturation factor [M p.110]

SF = clamp(1 + (MSI* − 50) ÷ 100 , 0.6, 1.8)

Symbol	Meaning	Where the value comes from
50, 100	Neutral point and scaling	[K] [M p.110]
0.6, 1.8	Lower and upper limits	[K] [M p.110]

Worked example (tags show where each number comes from)

- 1 + (46 − 50) ÷ 100 = 1 − 0.04 = **0.96** [EX]. Slightly open market, so slightly faster ramp-up.

Result: SF = 0.96 (example)

Why it matters: Above 50, the factor is above 1 and lengthens ramp-up. Below 50, it shortens it.

Equation 18 · Ramp-up period [M p.110]

R = max(3 , (24 − 1.8 × V_m) × SF)

Symbol	Meaning	Where the value comes from
24, 1.8	Base months and effect of market viability	[K] [M p.110]
3	Minimum ramp-up in months	[K] [C: training_report.json] minimum_ramp_months = 3
V_m, SF	From Eq 16 and Eq 17	[R]

Worked example (tags show where each number comes from)

- (24 − 1.8 × 5.4) × 0.96 = (24 − 9.72) × 0.96 = 14.28 × 0.96 = **13.7 months** [EX].

Result: R = 13.7 months (example)

Why it matters: The number of months before the business is expected to cover its own running costs.

Equation 19 · Required daily sales [M p.110]

RDS = MFC ÷ (p̄ × m × 26)

Symbol	Meaning	Where the value comes from
MFC	Monthly fixed cost (Eq 14)	[R] ₱38,680
p̄	Average price of the plan's products or services	[IN] Price list; [EX] ₱100
m	Gross margin: the share of each sale kept as profit	[K] 40% [C: training_report.json] gross_margin = 0.4 [M p.110]
26	Working days per month	[K] [C: training_report.json]

Worked example (tags show where each number comes from)

- Profit on each sale = ₱100 × 0.40 = ₱40 [R].
- One sale a day for 26 working days earns 26 × ₱40 = ₱1,040 a month [R].
- To cover ₱38,680 a month: 38,680 ÷ 1,040 = **37.2 sales per day** [EX].

Result: RDS = 37.2 sales per day (example)

Why it matters: The number of sales a day the business must make just to cover its fixed costs.

Equation 20 · Daily sales capacity [M p.111]

DSC = RPB ÷ 7

Symbol	Meaning	Where the value comes from
RPB	Residents per business (Eq 13)	[R] 625
7	Days in a week: each resident buys about once a week	[K] [C: training_report.json] purchases_per_resident_per_day = 1/7 (0.142857)

Worked example (tags show where each number comes from)

- 625 ÷ 7 = **89.3** customers per day [EX].

Result: DSC = 89.3 per day (example)

Why it matters: The number of customers the area can plausibly support each day. Compare with RDS: 37 needed versus 89 possible means there is room.

Group 7: Plan Viability Score (Eq 21 to 23)

Equation 21 · Plan viability scorecard [M p.112]

$S = 100 \times \Sigma_{k=1..7}\, w_k \times C_k$

Symbol	Meaning	Where the value comes from
w_k	Weights of the 7 components: 0.40 market, 0.20 capital, 0.10 price, 0.08 experience, 0.07 staffing, 0.07 offering, 0.08 differentiation. They add to 1.00	[K] [C: training_report.json] component_weights; [M Table 28, p.112]
C_k	Score of each component, from 0 to 1	[R] Rules in Table 28 (see note below)
S	Scorecard total from 0 to 100	[R]

Worked example (tags show where each number comes from)

- Example: every component scores 0.5, so S = 100 × 0.5 = **50** [EX].
- Component C1 (market) = 1 − MSI*/100 = 1 − 0.46 = 0.54 [R; Table 28].
- Component C2 (capital) = min(1, RW ÷ R) = min(1, 12.9 ÷ 13.7) = 0.94 [R].

Result: S = 50 (example)

Why it matters: Combines market, money, experience and idea into one score, like a report card with weighted subjects.

Equation 22 · Plan Viability Index training label [M p.112]

PVI* = clip(S + ε , 0, 100)

Symbol	Meaning	Where the value comes from
S	Scorecard total (Eq 21)	[R]
ε	Noise: normal, mean 0, standard deviation 3	[K] [C: training_report.json] label_noise_sd_points = 3; [M p.112]

Worked example (tags show where each number comes from)

- Scorecard S = 50 plus noise of about ±3, so the label is about **50** [EX].

Result: PVI* ≈ 50 (example)

Why it matters: The second forest learns to predict this label from 14 plan features. It is used for training only.

Equation 23 · Plan Viability Score (shown to the owner) [M p.112]

PVS = PVI ÷ 10

Symbol	Meaning	Where the value comes from
PVI	Predicted Plan Viability Index (0 to 100) from the trained plan model	[R] Second forest, 100 trees [C: training_report.json] plan_model
PVS	Score shown to the owner, from 0 to 10	[R]

Worked example (tags show where each number comes from)

- PVI = 65 [EX] ÷ 10 = **6.5 out of 10**.

Result: PVS = 6.5 (example)

Why it matters: The number the owner sees on Home and Recommendations.

Group 8: Break-even range (Eq 24 to 26)

Equation 24 · Break-even midpoint [M p.113]

$BE_{mid} = (24 - 1.8 \times PVS) \times SF$

Symbol	Meaning	Where the value comes from
PVS	Plan Viability Score (Eq 23)	[R] 6.5
SF	Saturation factor (Eq 17)	[R] 0.96

Worked example (tags show where each number comes from)

- (24 − 1.8 × 6.5) × 0.96 = (24 − 11.7) × 0.96 = 12.3 × 0.96 = **11.8 months** [EX].

Result: BE_{mid} = 11.8 months (example)

Why it matters: A strong plan in an open market gets a short break-even; a weak plan in a crowded market gets a long one. Manuscript example: PVS 8.0 at MSI 50 gives 9.6 months [M p.113].

Equation 25 · Lower bound of break-even [M p.113]

$BE_{low} = \max(3 , \text{round}(0.8 \times BE_{mid}))$

Symbol	Meaning	Where the value comes from
0.8	Lower band factor	[K] [M p.113]
3	Minimum months	[K] [M p.113]

Worked example (tags show where each number comes from)

- round(0.8 × 11.8) = round(9.44) = 9. max(3, 9) = **9 months** [EX].

Result: BE_{low} = 9 months (example)

Why it matters: The optimistic end of the range.

Equation 26 · Upper bound of break-even [M p.113]

$BE_{high} = \max(BE_{low} + 1 , \text{round}(1.25 \times BE_{mid}))$

Symbol	Meaning	Where the value comes from
1.25	Upper band factor	[K] [M p.113]

Worked example (tags show where each number comes from)

- round(1.25 × 11.8) = round(14.75) = 15. max(10, 15) = **15 months** [EX].

Result: BE_{high} = 15 months (example)

Why it matters: The range shown to the owner is 9 to 15 months. The upper bound is always at least one month above the lower bound.

Group 9: Quarterly outlook (Eq 27 to 29)

Equation 27 · Projected competitor count [M p.114]

$cc_q = \text{round}(cc_0 \times G_q)$

Symbol	Meaning	Where the value comes from
cc_0	Current competitor count	[IN] [EX] 15
G_q	Growth factor for quarter q (see the correction in Eq 28)	[R]

Worked example (tags show where each number comes from)

- Using the corrected growth factor from Eq 28 below, with 2 quarters ahead: round(15 × 1.0609) = **16** [EX].

Result: cc_2 = 16 (example)

Why it matters: The competitor count is the one input expected to change most over the next year.

Equation 28 · Competitor growth index [M p.114]

$G_q = (1 + g)^q - 1$ (as printed in the manuscript)

Symbol	Meaning	Where the value comes from
g	Quarterly growth rate: observed from the barangay's own history (limited to ±15%) or from national MSME counts (PSA and DTI, 2019 to 2023, then 3% a year) [M p.114]	[IN] / [K]
q	Quarter number	[K]

Worked example (tags show where each number comes from)

- **Correction (important).** As printed, the formula gives G = 0 for q = 1, so the first quarter would not be 'the present' as the text says.
- The code multiplies the current count by (1 + rate) raised to the quarters ahead of now, so the present quarter has a factor of 1. [C: app/services/saturation_timeline_service.py lines 206 to 212].
- Corrected reading used here: factor = (1 + g) ^ (quarters ahead). Example with g = 3% and 2 quarters ahead: $1.03^2 = 1.0609$ [EX].

Result: printed formula does not match the text or the code

Why it matters: Fix the printed Eq 27 and 28 in the manuscript before the defence so that the equation matches the code.

Equation 29 · Confidence of a quarterly forecast [M p.115]

$$\mathbf{Conf_q = \max(\ 0\ ,\ \min(Conf_{market},\ Conf_{plan}) - 8 \times (q - 1)\ )}$$

Symbol	Meaning	Where the value comes from
$Conf_{market}$, $Conf_{plan}$	Confidence of the two forests for that quarter	[R] Eq 6 for each model
8	Points lost per quarter ahead	[K] [M p.115]; [C: saturation_timeline_service.py] HORIZON_CONFIDENCE_PENALTY

Worked example (tags show where each number comes from)

- If the lower confidence is 72 [EX]: quarter 1 = 72, quarter 2 = 72 − 8 = 64, quarter 3 = 56, quarter 4 = **48**.

Result: 72, 64, 56, 48 (example)

Why it matters: The outlook gets less certain the further it looks ahead.

Group 10: How accurate the model is (Eq 30 to 33)

These figures come from the **synthetic test data**, not from real businesses. They show how closely the models copy their own formulas. The training report says so directly: "Re-train on real DTI/PSA/LGU historical data before citing these numbers as real-world accuracy" [T].

Equation 30 · Mean Absolute Error (MAE) [M p.115]

$$\mathbf{MAE = (1 \div n) \times \Sigma\ |y_i - \hat{y}_i|}$$

Symbol	Meaning	Where the value comes from
y_i	True label of test record i (from Eq 1 or Eq 22)	[R] Synthetic
$\hat{y}_i$	Model prediction for record i	[R] Forest output
n	Number of test records: 800 (market model), 1,200 (plan model)	[C: training_report.json] test_samples

Worked example (tags show where each number comes from)

- Errors of 2, 4 and 3 points: (2 + 4 + 3) ÷ 3 = **3.0** [EX].
- Real results: market model MAE = **3.371** points; plan model MAE = **3.139** points [T].

Result: MAE 3.37 (market), 3.14 (plan) [T]

Why it matters: A typical miss of about 3 points on a 0 to 100 scale.

Equation 31 · Root Mean Square Error (RMSE) [M p.115]

$$\mathbf{RMSE = \sqrt{(\ (1 \div n) \times \Sigma\ (y_i - \hat{y}_i)^2\ )}}$$

Symbol	Meaning	Where the value comes from
same as Eq 30	Symbols as above	[R]

Worked example (tags show where each number comes from)

- Squared errors 4, 16 and 9 average (4 + 16 + 9) ÷ 3 = 9.67; √9.67 = **3.1** [EX].
- Real results: market model RMSE = **4.219**; plan model RMSE = **3.930** [T].

Result: RMSE 4.22 (market), 3.93 (plan) [T]

Why it matters: Squaring gives extra weight to big misses, so RMSE is always at least as large as MAE.

Equation 32 · Coefficient of determination (R^2) [M p.116]

$$\mathbf{R^2 = 1 - [\ \Sigma\ (y_i - \hat{y}_i)^2\ ] \div [\ \Sigma\ (y_i - \bar{y})^2\ ]}$$

Symbol	Meaning	Where the value comes from
$\bar{y}$	Mean of the true labels	[R]

Worked example (tags show where each number comes from)

- Real result: plan model R^2 = **0.9086**, shown as 0.91 [T].
- Meaning: the model explains about 91% of the ups and downs in the plan labels.

Result: R^2 = 0.91 (plan model) [T]

Why it matters: A value close to 1 means the model follows the labels closely. It still says nothing about real businesses.

Equation 33 · Accuracy percentage [M p.116]

Accuracy = 100 − MAE

Symbol	Meaning	Where the value comes from
MAE	From Eq 30	[T]

Worked example (tags show where each number comes from)

- Market model: 100 − 3.371 = **96.63%** [T].
- Plan model: 100 − 3.139 = **96.86%** [T].

Result: 96.63% and 96.86% [T]

Why it matters: This is a closeness score, not classification accuracy. The target was 85% [T].

Group 11: Survey statistics (Eq 34 and 35)

These describe the usability questionnaire answered by 21 SME owners [M p.132 to 133], not the forecast.

Equation 34 · Weighted mean [M p.132]

WM = Σ (f × x) ÷ N

Symbol	Meaning	Where the value comes from
f	Number of respondents who chose a rating	[IN] Questionnaire
x	The rating, 1 (Strongly Disagree) to 5 (Strongly Agree)	[IN] Questionnaire
N	Total number of responses	[IN] Questionnaire

Worked example (tags show where each number comes from)

- Five people rated 5, three rated 4, two rated 3 [EX]: (5×5 + 3×4 + 2×3) ÷ 10 = (25 + 12 + 6) ÷ 10 = **4.3**.

Result: WM = 4.3 (example)

Why it matters: Gives each answer a weight equal to how many people chose it.

Equation 35 · Grand mean [M p.133]

GM = Σ WM ÷ n

Symbol	Meaning	Where the value comes from
WM	Weighted mean of each quality characteristic (Eq 34)	[R]
n	Number of quality characteristics: 6 (ISO/IEC 25010)	[M p.ii, p.133]

Worked example (tags show where each number comes from)

- Two qualities scored 4.3 and 4.6 [EX]: (4.3 + 4.6) ÷ 2 = **4.45**.
- Real result: grand mean across six characteristics = **4.59**, 'Excellent' (range 4.21 to 5.00) [M p.ii].

Result: GM = 4.59 (real, usability) [M]

Why it matters: A single usability score for the whole system.

D2A. Software development requirements, explained

Section 3.2.1, Table 3 (manuscript pp.36 to 38) lists the software used to build and run MarketLens. This table adds the exact pinned version and where each tool appears in the project. Evidence tags show how the claim was checked.

Evidence tags for this section

Tag	Meaning
[C]	Verified by reading the file in the project folder (path given)
[D]	Stated in DEPLOYMENT.md in the project folder
[M]	Stated only in the manuscript (Table 3); not checked in the code

Requirement	What it is used for	Evidence (where it is in the project)	Plain-language analogy
Operating system	Windows 11 (development) and Linux on Render (live)	[M p.36] [C: render.yaml]	Your computer runs the code; Render's Linux servers run it online.
Python	3.14 (.python-version)	[D] DEPLOYMENT.md; [M p.37]	Language of the whole backend and the forecasting engine.
Flask 3.0.3, Werkzeug 3.0.4	Web framework: routes, page rendering, MVC	[C: requirements.txt] [C: app/__init__.py] create_app	The waiter: takes the request and sends it to the right controller.
Flask-SQLAlchemy 3.1.1, PyMySQL 1.1.1	Maps Python classes to database tables and connects to MySQL	[C: requirements.txt] [C: app/__init__.py] imports db from app/extensions.py; models/ folder	Lets the code read and write tables without raw SQL everywhere.
MySQL 8.0 (local) and Aiven (live)	Database: users, plans, market and LGU data, forecasts, audit logs	[C: sql/schema.sql] [C: certs/aiven-ca.pem]; [M p.37]	The ledger that remembers everything.
cryptography 43.0.1	Needed by PyMySQL for MySQL's login method; also encrypts personal data	[C: requirements.txt comment]; [M p.37]	The secure lock on the ledger's connection.
Flask-Login 0.6.3	Login sessions and role checks	[C: requirements.txt] [C: app/extensions.py] login_manager	The ID check at the door.
Flask-WTF 1.2.1	CSRF protection on forms	[C: requirements.txt] [C: app/extensions.py] CSRFProtect	Stops other websites from submitting forms on the user's behalf.
Flask-Caching 2.x	Stores slow results (city-wide trend reports) for a while	[C: requirements.txt] [C: app/extensions.py] Cache	Keeps yesterday's answer on the shelf so it is not recomputed each visit.
scikit-learn 1.9.0	K-Means, Random Forest, and the accuracy measures	[C: requirements.txt] [C: app/ml/train_model.py] lines 77 to 80	The recipe that makes the predictions.
NumPy 2.5.3	Number arrays used by the models	[C: requirements.txt] [C: app/services/forecasting_service.py] import numpy	The measuring cups.
pandas 3.0.5	Tables of data for cleaning and organizing	[C: requirements.txt]; [M p.37]	The data sheets that organize the ingredients.
joblib 1.4.2	Saves and loads the trained model files	[C: requirements.txt] [C: app/services/forecasting_service.py] line 58; [C: app/ml/model_store/]	The freezer: store the trained model, reload it fast.
openpyxl 3.1.5	Reads Excel uploads from LGU officers	[C: requirements.txt]; [M p.37]; [C: sample_data/*.xlsx]	Opens the permit register spreadsheet.
contourpy 1.3.3	Draws the barangay cells on the saturation map	[C: requirements.txt]; [M p.37]	Cuts the city map into the right shapes.
requests 2.32.3, httpx (0.27 line)	Calls outside APIs over the internet	[C: requirements.txt]; [C: app/services/places_service.py] calls places.googleapis.com	The courier that goes out to other services.
python-dotenv 1.0.1	Loads secret keys from the .env file	[C: requirements.txt]; [C: .env file exists]; [M p.37]	The safe-deposit key kept outside the code.

Requirement	What it is used for	Evidence (where it is in the project)	Plain-language analogy
Google Places API	Counts competitors per barangay and industry	[C: app/services/places_service.py] (Text Search)	The market surveyor who counts the shops.
Google Gemini (OpenAI, Anthropic optional)	Writes plain-language explanations of forecasts	[C: app/services/llm_service.py] (Gemini is the default)	The narrator who explains the numbers in words.
Brevo e-mail API	Sends verification and reset codes	[C: app/services/email_service.py] (BREVO_API_KEY)	The mail carrier.
gunicorn 23+ (live), waitress 3.0.0 (fallback)	Serves the site instead of Flask's development server	[C: Procfile] and [C: render.yaml] startCommand; [C: requirements.txt]	The staff who actually serve customers.
Render	Hosts the web app	[C: render.yaml]; [D] DEPLOYMENT.md	The restaurant building that is open to the public.
Aiven MySQL	Hosts the production database	[C: certs/aiven-ca.pem]; [D] DEPLOYMENT.md	The secure off-site warehouse.
Git and GitHub	Version history; each push to main redeploys on Render	[C: .git folder]; [M p.37]	The recipe book with its full edit history.
PyCharm 2026.2.3	Code editor	[C: .idea folder]; [M p.38]	The chef's workbench.
pytest 8.3.2, Node.js	Automated tests (Python and JavaScript)	[C: requirements.txt]; [C: tests/ folder, about 45 test files and 2 JavaScript tests]	Taste tests before a dish goes out.
Bootstrap 5.3.3 and Bootstrap icons	Responsive layout and icons	[C: templates use bi bi-* icons, e.g. app/templates/lgu/dashboard.html]; version [M p.37]	The plate design that fits any screen size.
Chart.js 4.4.4	Trend charts	[C: canvas charts in app/templates/sme/trend_reports.html]; version [M p.37]	The graph paper.
Leaflet 1.9.4 with OpenStreetMap	Interactive saturation map	[C: app/static/js/map.js]; version [M p.37]	The interactive city map on the wall.

How the software works together: one click

An SME presses **Generate Forecast**. Each step below names the tool and the file that does it.

1. Flask routes the request to a controller after checking the login and CSRF token [C: app/extensions.py].
2. If competitor counts are missing, Google Places is called (live lookups are switched off on the live site) [C: places_service.py].
3. scikit-learn models, loaded from joblib files, compute the saturation index and the viability score [C: forecasting_service.py; train_model.py].
4. The result is saved in MySQL through SQLAlchemy as one forecast record [C: models/forecast_result.py; sql/schema.sql].
5. Gemini writes the explanation. The code checks that every number in the text matches the computed numbers, and falls back to a rule-based writer if it does not [C: llm_service.py].
6. Chart.js and Leaflet draw the trends and the map in the browser [C: templates/sme/trend_reports.html; static/js/map.js].
7. Gunicorn serves all of this on Render [C: Procfile; render.yaml].

System questions S1A to S6A

S1A. What is the purpose of the trend reports for LGU and SME users?

The Trend Reports page is a single city-wide report shared by SME, LGU and Admin accounts. The code comment says an earlier version gave SMEs a separate per-plan report, which was removed [C: app/services/trend_analytics_service.py]. Each role reads it for a different purpose:

SME owners judge the market before committing capital: whether their industry is getting more crowded over time, how demand is moving, and whether now is a good time to open. Their own plan's figures stay on Home and Recommendations.

LGU officers plan locally: which industries are growing or saturating across barangays, to decide where to encourage or limit business. The report can be printed for a chosen month.

Two caveats: months before the app's own records are estimated from the national MSME counts (DTI/PSA, 2019 to 2023) and labelled "(estimated)". The app's market figures come from Google Places snapshots, so they are not live.

S2A. What is behind the procedure and computation of the output after an SME enters business parameters?

- **Competitor count.** Same-industry businesses in the barangay, from stored Google Places data [C: forecasting_service.py].
- **Market snapshot.** Population density, foot traffic, average rent and survival rate [C: forecasting_service.py docstring].
- **Business density.** From a real LGU upload if one exists, otherwise an automatic placeholder row.
- **Saturation index (MSI).** The first Random Forest averages 100 trees over eight features (Eq 5, p.104).
- **Level label.** Low, Moderate, High or Saturated, from fixed cut points (Eq 9).
- **Confidence.** How closely the trees agree (Eq 6). It is not a probability of success.
- **Sub-category adjustment.** If the plan names a niche, the count is scaled by measured niche competition (Eq 11 and 12), giving MSI*.
- **Plan finances.** Residents per business, monthly fixed cost, capital runway, ramp-up months, and required daily sales (Eq 13 to 20).
- **Plan Viability Score.** A second Random Forest with 14 plan features predicts the viability out of 10 (Eq 21 to 23).
- **Break-even range and quarterly outlook.** Eq 24 to 29.
- **Storage and explanation.** One forecast record is saved. An early-warning notification is sent if the market crosses the threshold. Gemini writes the explanation; a rule-based writer is the fallback.

The business inputs drive the result: more capital and staff raise the plan's score, while higher rent and a crowded market lengthen ramp-up and break-even.

S3A. Why does Render always load, redeploy or wake the application before access?

Your Render app is on the free tier, which causes a sleep and wake cycle.

- **Spin-down.** A free Render service sleeps after 15 minutes with no traffic, and the next visitor wakes a fresh container, which takes about a minute [D] DEPLOYMENT.md.
- **Redeploy is a different event.** Each push to the main branch on GitHub triggers a full redeploy [M p.37].
- **App startup.** The app loads about 210 MB of trained models into a 512 MB instance with less than one CPU [D]. warmup.py does the loading and template compilation before the port opens, so visitors do not meet a half-ready process [C: app/services/warmup.py]. The first city-wide AI sweep takes about 15 seconds, then it is cached.
- **Fixes.** Open the site about five minutes before presenting and leave the tab open (the guide's advice). An uptime pinger also works, but 750 free instance hours a month is roughly one service running all month. As far as I know, paid Render plans do not sleep this way; that is not in your docs, so check before relying on it.

S4A. Why does the LGU recommendation have no record yet?

City-wide recommendations are blocked until an LGU officer uploads real data. The code (has_active_lgu_data) ignores automatic placeholder rows, so the app does not rank barangays using figures nobody submitted [C: app/services/data_import_service.py]. The dashboard therefore shows: "No official LGU dataset is active. Upload business permit data to generate city-wide recommendations" [C: app/templates/lgu/dashboard.html].

Two causes are possible:

No permit register has been uploaded yet. Use the Government Data Upload page with an Excel or CSV file. A sample is in sample_data/ for the defence.

An upload happened but its barangay names did not match the official list of 76. Unmatched names are rejected and shown with suggestions. The saturation map and trend reports still work without an upload.

I cannot see the live database, so I cannot tell which applies to your live site.

S5A. How accurate is the system?

The honest answer: its accuracy against real businesses has not been measured.

• **What was measured.** The models were trained and tested on synthetic data generated from the system's own formulas. Market model: MAE 3.37 and RMSE 4.22 points on 0 to 100 (96.63%). Plan model: MAE 3.14, RMSE 3.93 and R^2 0.91 (96.86%). Both beat the 85% target [T]. This shows how well the models reproduce their own formulas, not how well they predict real outcomes.

• **Caveat on the 'accuracy' figure.** It is 100 minus MAE, a closeness score, not classification accuracy (Eq 33).

• **Built-in caveat.** The training report and the manuscript both say the models must be retrained on real DTI, PSA and LGU outcomes before real-world accuracy is claimed [T] [M p.117].

• **Other inputs are approximate.** The 2024 PSA population, the 2026 Google Places snapshot and the wage assumptions are estimates. The scoring weights are judgment calls.

• **What user testing showed.** Usability by 21 SME owners scored 4.59 out of 5 ('Excellent'). Two IT experts scored 3.975 ('Very Good'). Black-box testing passed all 18 test cases [M]. None of these measures forecast accuracy.

For the defence, present the 96% figures as 'fidelity to the documented model', and say that real-world validation is future work.

S6A. What does each file and folder contain, and why is the project layered?

The project is a single Flask application in layers, following the Model-View-Controller pattern and the three-layer design in manuscript §3.1 (pp.33 to 36) [M].

Folder or file	What is inside	Why it is there
app.py, wsgi.py	Local start point; the production entry point gunicorn uses	Two ways in: app.py locally, wsgi.py on Render [C]
Procfile, render.yaml	Start command and Render service settings	Tell Render how to run the app [C]
requirements.txt, .python-version	Pinned library list and Python version	Rebuild the same environment [C]
seed.py	Loads the 76-barangay reference data and trains the models	First-time setup [C]
app/__init__.py, config.py, extensions.py	App factory; settings from .env; shared database, login, CSRF and cache objects	Central setup [C]
app/controllers/ (Controller)	Route handlers for admin, auth, SME, LGU, profile, onboarding, community, API	Receive requests, check who is asking, hand work to services [C]
app/models/ (Model)	One Python class per database table	The data layer's shape [C]
app/services/	Business logic: forecasting, plan viability, recommendations, trend analytics, diversification, Gemini, e-mail, Places, warm-up	The brain the controllers call [C]
app/ml/ and model_store/	Training code; saved model files and the training report	Keeps machine learning apart from web code [C]
app/templates/ (View), app/static/	HTML pages per role; CSS, JavaScript, images, map boundary data	Presentation layer [C]
app/utils/	Audit trail, privacy, decorators, performance helpers	Shared tools used across layers [C]

Folder or file	What is inside	Why it is there
sql/	schema.sql plus dated migration scripts	Records how the database changed over time [C]
tests/	About 45 Python test files and 2 JavaScript tests	Checks each layer [C]
scripts/	One-off maintenance tools (API checks, data push, cache warm-up)	Not needed to run the app [C]
Reference/	CAPSTONE_ALIGNMENT.md, DATASETS.md, FORECAST_MODEL.md	Links the code to the manuscript [C]
sample_data/	Sample LGU upload in CSV and Excel	Demos and upload tests [C]
certs/aiven-ca.pem	Certificate for the encrypted connection to Aiven MySQL	Needed by the hosted database [C]
instance/	Generated caches and uploads	Runtime files, not source code [C]
DEPLOYMENT.md, README.md	Setup and deployment guides	Documentation [C]
.env, .env.example	Real secret settings; blank template	Keep secrets out of the code [C]
venv/, .idea/, .pytest_cache/, .git/	Local Python environment; PyCharm settings; test cache; version history	Tooling, not part of the product [C]

Why it is layered. The manuscript says the system runs as one app rather than three programs talking through middleware, which keeps deployment simple [M p.34]. The layers separate responsibilities:

- The **view** (templates) only shows things, so a chart can change without touching the calculations.
- The **controller** only checks users and routes requests, so the security rules sit in one place.
- The **services** hold the forecasting logic, so a model can be retrained without rebuilding pages.
- The **model** (data layer) only stores and retrieves data.

Each layer passes on only what the next one needs, so one part can change without breaking the others, and the tests can target one layer at a time.

Scope of this export. This file was generated from the Q&A answers only. The project folder was read, not changed. The pinned chat could not be read from here.